

RV Fotos

USUARIO

esInnecesaria :: RV -> CUIF -> Bool

esInnecesaria rv cuif = isEmptyS (quienesAparecen rv cuif)

La eficiencia es $O(\log F)$ ya que es el costo de la función quienesAparecen

juntos :: Vehiculo -> Vehiculo -> RV -> Set CUIF

juntos v1 v2 rv = if aparecenJuntos rv v1 v2

then enQueFotosAparecen rv (todasLasFotos rv) v1 v2

else emptyS

enQueFotosAparecen :: RV -> [CUIF] -> Vehiculo -> Vehiculo -> set CUIF

enQueFotosAparecen _ [] _ = emptyS

enQueFotosAparecen rv (c:cs) v1 v2 = let vehiculos = quienesAparecen rv c

if belongsS v1 vehiculos && belongsS v2 vehiculos

then addS c (enQueFotosAparecen rv cs v1 v2)

else enQueFotosAparecen rv cs v1 v2

La eficiencia es $O(F * (\log F + \log vf))$ ya que F es la cantidad de fotos del registro y por cada foto se hace la tarea quienesAparecen que tiene un costo $\log F$ y se aplica la función belongsS que tiene un costo de $O(\log vf)$

data RV = MkRV (Map CUIF Foto) --fotos del registro MapCF

(Map Vehiculo (Set Vehiculo)) --Vehiculos que aparecen juntos en alguna foto
MapV

INV. REP:

* Por cada clave c en mcf su valor asociado Vehiculo debe tener como CUIF dicha clave c .

* Si un vehiculo aparece en alguna foto de mcf, debe ser clave en mvvs.

* Si un vehiculo $v1$ aparece con otro $v2$, entonces $v2$ también aparece con $v1$.

* Ninguna clave c en mcf aparece dentro de su valor asociado Set Vehiculo.

agregarFoto :: RV -> Foto -> RV

agregarFoto (MkRV mf mv) (MkF c vehs) =

case lookupM c mf of

Just _ -> MkRV mf mv -- ya existe la foto, no se modifica

Nothing ->

let nuevoMF = assocM c (MkF c vehs) mf

listaVehs = set2list vehs

```

    nuevoMV = actualizarCoapariciones listaVehs mv
  in MkRV nuevoMF nuevoMV

-- Actualiza el mapa de coapariciones, recursivamente
actualizarCoapariciones :: [Vehiculo] -> Map Vehiculo (Set Vehiculo) -> Map Vehiculo (Set Vehiculo)

actualizarCoapariciones [] mv = mv
actualizarCoapariciones (v:vs) mv =
  let mv' = agregarPares v vs mv
  in actualizarCoapariciones vs mv'

-- Agrega simétricamente los pares (v, v2) para cada v2 en la lista dada
agregarPares :: Vehiculo -> [Vehiculo] -> Map Vehiculo (Set Vehiculo) -> Map Vehiculo (Set Vehiculo)

agregarPares _ [] mv = mv
agregarPares v (v2:resto) mv =
  let mv' = agregarPar v v2 mv
  in agregarPares v resto mv'

-- Agrega relación simétrica v1-v2
agregarPar :: Vehiculo -> Vehiculo -> Map Vehiculo (Set Vehiculo) -> Map Vehiculo (Set Vehiculo)

agregarPar v1 v2 mv =
  let set1 = case lookupM v1 mv of
    Nothing -> addS v2 emptyS
    Just s -> addS v2 s
    mv1 = assocM v1 set1 mv
    set2 = case lookupM v2 mv1 of
    Nothing -> addS v1 emptyS
    Just s -> addS v1 s
  in assocM v2 set2 mv1

```

Para una foto con vf vehículos:

`actualizarCoapariciones` recorre la lista $\rightarrow O(vf)$

Cada llamada a `agregarPares` recorre los demás vehículos $\rightarrow O(vf)$

En cada par se hacen operaciones sobre Map y Set $\rightarrow O(\log vf)$

Total: $O(vf^2 * \log vf)$, tal como exige el enunciado.

todasLasFotos :: RV -> [CUIF]

```
todasLasFotos (MkRV mcf _) = keysM mcf
```

```
quienesAparecen :: RV -> CUIF -> Set Vehiculo
```

```
quienesAparecen c (MkRV mcf _) = case (lookupM c mcf) of
```

```
    Nothing -> error "El CUIF no corresponde a una foto de RV"
```

```
    Just (MkF c sv) -> sv
```

```
aparecenJuntos :: RV -> Vehiculo -> Vehiculo -> Bool
```

```
aparecenJuntos (MkRV _ mvs) v1 v2 = if v1 /= v2
```

```
    then case lookupM v1 mv of
```

```
        Nothing -> False
```

```
        Just setV -> belongsS v2 setV
```

```
    else error "Los vehículos tienen que ser diferentes"
```

Parte 3)

```
data RV = MkRV (Map CUIF Foto)      -- Fotos del registro
```

```
    (Map Vehiculo (Set Vehiculo))-- Coapariciones
```

```
    (Heap (Int, CUIF))      -- Heap ordenada por cantidad de vehículos (máximo arriba)
```

```
masCargada :: RV -> Maybe CUIF
```

```
masCargada (MkRV _ _ heap) = if isEmptyH heap
```

```
    then Nothing
```

```
    else Just (snd (findMaxH heap))
```

Busqueda

TAD Busqueda: data Busqueda = B (Map String (Map String Int)) - - nombre producto x atributos [Filtro] - - filtros aplicados a la búsqueda

```
siguientesN :: Búsqueda -> Int -> [(String,Int)]
```

```
siguientesN _ 0 = []
```

```
siguientesN búsqueda n = case siguiente búsqueda of
```

```
    (Just producto, nuevaBusqueda) -> producto : siguientesN
```

```
nuevaBusqueda (n-1)
```

```
    (Nothing,_) -> []
```

```
registrar :: String -> Int -> Map String Int -> Búsqueda -> Búsqueda
```

```
registrar prod pre atributos (B mpa fs) = case (lookupM prod mpa) of
```

```
    Nothing -> agregarProducto prod pre atributos mpa
```

```
    fs
```

```
    Just p -> B mpa fs
```

agregarProducto :: String -> Int -> Map String Int -> Map String (Map String Int) -> [Filtro] -> Busqueda

agregarProducto prod pre atributos mpa fs = let atributosNew = assocM "precio" pre atributos

if aplicarFiltros fs atributos

then B (assocM prod atributosNew mpa) fs

else B mpa fs

aplicanFiltros :: [Filtro] -> Map String Int -> Bool

aplicanFiltros [] = True

aplicanFiltros (f:fs) mpa = aplica f mpa && aplicanFiltros fs mpa

filtrar :: Filtro -> Búsqueda -> Búsqueda

filtrar filtro (B mpa fs) = B (aplicarFiltro filtro (keysM mpa) mpa) (filtro:fs)

aplicarFiltro :: Filtro -> [Producto] -> Map String (Map String Int) -> Map String (Map String Int)

aplicarFiltro f [] mpa = emptyM

aplicarFiltro f (p:ps) mpa = let atributos = fromJust(lookupM p mpa)

in if aplica f atributos

then assocM p atributos (aplicarFiltro f ps mpa)

else aplicarFiltro f ps mpa

data Investigacion = ConsI (Map Nombre Persona) MapP

(Map Evidencia [Nombre]) MapE

(PriorityQueue Persona) PQ

Int Ne

Investigacion

La eficiencia es de costo $O(1)$ ya que la operacion es simplemente mostrar el entero que representa la cantidad de evidencia.

esSospechoso :: Nombre -> Investigacion -> Bool

esSospechoso nom (ConsI mp _ _ _) = case (lookUpM nom mp) of

Nothing -> False

Just p -> cantEvidencia p >= 1

La eficiencia es $O(\log N)$ ya que el costo de lookUpM es logaritmico siendo N la cantidad total de personas en el mp.

posiblesInocentes :: Investigacion -> [Persona]

```
posiblesInocentes (ConsI mp _ _ _) = let ps = domM mp in
```

```
    posiblesInocentes' ps mp
```

```
posiblesInocentes' :: [Nombre] -> Map Nombre Persona -> [Persona]
```

```
posiblesInocentes' [] _ = []
```

```
posiblesInocentes' (n:ns) mp = let p' = fromJust(lookUpM n mp) in
```

```
    if cantEvidencia p' <= 0
```

```
    then p' : posiblesInocentes' ns mp
```

```
    else posiblesInocentes' ns mp
```

La eficiencia es $O(N \log N)$ ya que domM tiene un costo de $O(N)$ siendo N todas las claves nombre del mp, como así también dentro de la subtaska posiblesInocentes'

la operación de recorrer la lista tiene un costo de $O(N)$ y por cada elemento se realiza la operación lookUpM de costo de $O(\log N)$.

```
ingresarPersonas :: [Nombre] -> Investigacion -> Investigacion
```

```
ingresarPersonas [] inv = inv
```

```
ingresarPersonas (n:ns) (ConsI mp me pq n') = let p = crearP n
```

```
    in ingresarPersonas ns (ConsI (assocM n p mp)
```

```
    me (insertPQ p pq) n')
```

La eficiencia es $O(N \log N)$ ya que por cada Nombre de la lista se hace la operación lookUpM de costo $O(\log N)$ siendo N la cantidad de nombres del mp.

```
ingresarEvidencia :: Evidencia -> Nombre -> Investigacion -> Investigacion
```

```
ingresarEvidencia e nom (ConsI mp me pq n) = let persona = fromJust(lookupM n mp)
```

```
    nombres = fromJust(lookupM e me)
```

```
    newPersona = agregarEvidencia e persona
```

```
    newPQ = actualizarPQ newPersona pq
```

```
    newNombres = nombres : (nombre
```

```
    newPersona)
```

```
    in ConsI (assocM (nombre newPersona)
```

```
    newPersona mp) (assocM e newNombres me) newPQ (n+1)
```

```
actualizarPQ :: Persona -> PriorityQueue Persona -> PriorityQueue
```

```
actualizarPQ p pq = let maxP = maxPQ pq
```

```
    in if (nombre p) == (nombre maxP)
```

```
    then insertPQ p (deleteMaxPQ pq)
```

```
    else insertPQ maxP (modificarPQ p(deleteMaxPQ pq))
```

La eficiencia es $O(N \log N)$ ya que por cada persona de la pq se ejecuta la funcion deleteMaxPQ

comenzarConPersonas :: [Nombre] -> Investigacion

```
comenzarConPersonas ns = let inv = comenzarInvestigacion in
    ingresarPersonas ns inv
```

La eficiencia es $O(N \log N)$ ya que se ingresa cada persona de la lista a la Investigacion

todosInocentes :: Investigacion -> Bool

```
todosInocentes inv = cantEvidencialIngresada inv == 0
```

terminaCerrado :: [(Evidencia,Nombre)] -> Investigacion -> Bool

```
terminaCerrado [] inv = casoCerrado inv
```

```
terminaCerrado ((e,n):ens) inv = terminaCerrado ens (ingresarEvidencia e n inv)
```

MVTree

```
data MVTree = MVT (Heap (int,Producto)) --vendidos (Map Categoria MVTree) --
subcategorías
```

$O(J^2 + J * (J \log SC + P \log P))$

esMasVendido :: Producto -> CaminoJ -> Int -> MVTree -> Bool

```
esMasVendido p cs n t = esMasVendido' p (inits cs) n t
```

$O(N * (J \log SC + P \log P) + P) = N * (J \log SC + P \log P)$

esMasVendido' :: Producto -> [CaminoJ] -> Int -> MVTree -> Bool

```
esMasVendido' p [] n t = False
```

```
esMasVendido' p (cs:css) n t = elem p (esMasVendidoEn n cs t) || esMasVendido' p css n t
```

$O(J * \log SC)$

registrarCategoria :: CaminoJ -> MVTree -> MVTree

```
registrarCategoria [] t = t
```

```
registrarCategoria (c:cs) (MVT h m) = case lookupM c m of
```

```
    Nothing -> MVT h (assocM c (registrarCategoria cs (MVT
emptyH emptyM)) m)
```

```
    Just t -> MVT h (assocM c (registrarCategoria cs t) m)
```

$O(J * \log SC)$

subCategoriasDe :: CaminoJ -> MVTree -> [Categoria]

```
subCategoriasDe [] (MVT h m) = domM m
```

```
subCategoriasDe (c:cs) (MVT h m) = case lookupM c m of
```

```
    Nothing -> error "Jerarquia invalida"
```

Just t -> subCategoriasDe cs t

O(J (log SC + P log P))

registrarVenta :: Producto -> CaminoJ -> MVTree -> MVTree

registrarVenta p [] (MVT h m) = MVT (registrarVentaH p h) m

registrarVenta p (c:cs) (MVT h m) = case lookupM c m of

Nothing -> error "jerarquia invalida"

Just t -> MVT (registrarVentaH p h) (assocM c
(registrarVenta p cs t) m)

registrarVentaH :: Producto -> Heap (Int, Producto) -> Heap (Int, Producto)

registrarVentaH p h = if isEmptyH h

then insertH (1,p) h

else let (i,p') = findMaxH h in

if p == p'

then insertH (i+1,p) (deleteMaxH h)

else insertH (i,p') (registrarVentaH p (deleteMaxH h))

masVendidosEn :: Int -> CaminoJ -> MVTree -> [Producto]

masVendidosEn i [] (MVT h m) = if i < 0

then []

else takeH i h

masVendidosEn i (c:cs) (MVT h m) = case lookupM c m of

Nothing -> error "jerarquia invalida"

Just t -> masVendidosEn i cs t

takeH :: Int -> Heap (Int, Producto) -> [Producto]

takeH 0 h = []

takeH n h = if isEmptyH h

then []

else (snd (findMaxH h)) : takeH (n-1) (deleteMaxH h)

EMgame

ganarE :: EmGame -> Personaje -> Personaje -> Esmeralda -> EmGame

ganarE (AG mpe memp h) p1 p2 e = let esm = lookupM e memp

pl1 = lookupM p1 mpe

pl2 = lookupM p2 mpe

in if isJust pl1 && isJust pl2

```

then case esm of
  Nothing -> error "la esm no esta en el juego"
  Just p -> case mp of
    Nothing -> error "la esmeralda no es de pl1
ni pl2"
    Just p -> Just p -> if p == pl1 || p == pl2
                                then if poder pl1 > poder
pl2
                                then AG (moverEsm mpe
e pl1 pl2) () h
                                else AG (moverEsm
mpe e pl2 pl1) () h
                                else error "la esm no es de
ningún jugador"
else error "Algun personaje no esta en el juego"

```

Reality

```

data Reality = R (Map Nombre Participante)
               (Map Juego [Nombre])
               (PriorityQueue Participante)
               Int

```

comenzarConParticipantes :: [Nombre] -> Reality

comenzarConParticipantes [] = comenzarReality

comenzarConParticipantes (n:ns) = ingresarParticipante n (comenzarParticipantes ns)

$O(P \log P)$

sonTodosNovatos :: Reality -> Bool

sonTodosNovatos r = lenght (losQueEstanAlHorno r) == cantParticipantes r

$O(P \log P)$

losSobrevivientes :: Reality -> [Participante]

losSobrevivientes r = if hayClaroGanador r

then unClaroGanador r : losSobrevivientes(sinUnClaroGanador r)

else []

$O(P * J(P + \log J))$

Siendo R mps mjs ps jn

*El valor P asociado a cada clave n en mps cumple que nombre p es n

*Si J es elemento de Juego P, siendo P un valor asociado a clave en mps entonces J es clave en mjs y nombre P pertenece al valor asociado al mismo.

*Las listas asociadas a cada clave J en mjs no tienen repetidos --> me falto

*Para cada P que sea valor asociado a una clave mps, P esta en PS

*Para cada P que esta en PS, P es valor a una clave en mps

*En PS no hay elementos repetidos --> me falto

*jn es la cantidad de claves en mps

ingresarParticipante :: Nombre -> Reality -> Relity

ingresarParticipante n (R mps mjs ps jn) = case lookupM n mps of

Nothing -> let participante = crearP n

in R (assocM n p mps)

mjs

(insertPQ p ps)

n

Just _ -> error "El participante ya existe en el

Reality"

losQueEstanAlHorno :: Reality -> [Participante]

losQueEstanAlHorno (R mps _ _ _) = losQueNoTienenJuegos (keys mps) mps

losQueNoTienenJuegos :: [Nombre] -> Map Nombre Participante -> [Participante]

losQueNoTienenJuegos [] mps = []

losQueNoTienenJuegos (n:ns) mps = case lookupM n mps of

Nothing -> error "no puede pasar"

Just p -> if cantJuegos p == 0

then p : losQueNoTienenJuegos ns mps

else losQueNoTienenJuegos ns mps

unClaroGanador :: Reality -> Bool

unClaroGanador (R _ _ ps _) = hayPrimeroConMasDe5 ps

hayPrimeroConMasDe5 :: PriorityQueue Participante -> Bool

hayPrimeroConMasDe5 ps = not(isEmptyPQ ps) && cantJuegos (maxPQ ps) >= 5

unClaroGanador :: Reality -> Participante

unClaroGanador (R _ _ ps _) = if hayPrimeroConMasDe5 ps

then maxPQ ps

else error "no hay un claro ganador"

sinElClaroGanador :: Reality -> Reality

```
sinElClaroGanador (R mps mjs ps jn) = if hayPrimeroConMasDe 5 ps
    then let p = maxPQ ps
         in R (deleteM (nombre p) mps)
              quitarParticipanteDe (juego p) (nombre p) mjs)
        (deleteMaxPQ p ps)
    jn
else error "No hay un claro ganador"
```

quitarParticipanteDe :: [Juego] -> Nombres -> Map Juego [Nombre] -> Map Juego [Nombre]

```
quitarParticipanteDe [] _ mjs = mjs
quitarParticipanteDe (j:js) n mjs = case lookupM j mjs of
    Just ns -> assocM J (borrar n ns) (quitarParticipanteDe js n
mjs)
    Nothing -> error "No puede pasar"
```

ordenados :: Tree Int -> [Int]

ordenados Empty = []

ordenados (NodeT v ti td) = ims v (comb(ordenados ti) (ordenados td))

type PersonaOrd = (Int,Persona)

Las tuplas se ordenan de izquierda a derecha

el orden se define por que pueden compararse por mayor, menor o igual

el orden en String esta definido por el orden de las letras en el abecedario

Map Persona [Persona]

--hacer eliminar personas que no tengan amigos--uno usando delete (podar) y otro usando assoc (reconstruir)--

MaxHeap(Int,Persona) Int es la cantidad de amigos, por lo que se ordena de mayor a menor en cantidad de amigos

agregarAmigoHeap :: Persona -> **MaxHeap** (Int,Persona) -> **MaxHeap** (Int,Persona)

```
agregarAmigoHeap p mh = if isEmptyMH mh
    then insertMH (1,p) mh
    else let (i,p') = findMaxH mh
         in if p' == p
            then insertMH (i+1,p) (deleteMaxMH mh)
```

```

else insertMH (i,p') (agregarAmigoHeap p (deleteMaxMH
mh))
type Coord = (Int, Int)
type Cell = (Int, Int, Int, Int)
data ChangeType = NoChange
                | ChangeCell Coord
                | HeadChange
data GBoard = GBB Coord (History Coord) (Map Coord (History Cell)) (History
ChangeType)
ponerEn :: Color -> Cell -> Cell
nroBolitasEn :: Color -> Cell -> Int
puedeMoverA :: Coord -> Dir -> Coord -> Bool
moverA :: Coord -> Dir -> Coord -> Coord
mover :: Dir -> GBoard -> GBoard
mover d (GBB size head cells changes) = let pos = current head
                                         in GBB size (register (moverA size d pos) head)
                                         cells (register HeadChange changes)
ponerBolitaRojaEnCadaCeldaDelTablero :: GBoard -> GBoard
ponerBolitaRojaEnCadaCeldaDelTablero gb =
ponerBolitasRojasYaInicializado(irAlBordeSur(irAlBordeOeste gb))
ponerBolitasRojasYaInicializado :: GBoard -> GBoard
ponerBolitasRojasYaInicializado gb = if (haySiguieteCeldaEnRecorridoAL_Y_ Este Norte
gb)
    then ponerBolitasRojasYaInicializado
(irASiguieteCeldaEnRecorridoAL_Y_ Este Norte (PonerRojo gb))
    else Poner Rojo gb
haySiguieteCeldaEnRecorridoAL_Y_ :: Dir -> Dir -> GBoard -> Bool
haySiguieteCeldaEnRecorridoAL_Y_ d1 d2 gb = puedoMover d1 gb || puedoMover d2 gb

irASiguieteCeldaEnRecorridoAL_Y_ :: Dir -> Dir -> GBoard -> GBoard
irASiguieteCeldaEnRecorridoAL_Y_ d1 d2 gb = if puedoMover d1 gb
    then mover d1 gb
    else irAlBorde (opuesto d1) (mover d2 gb)
el costo es  $O(N*M)$  siendo N la cantidad de celdas del tablero y M el ancho del tablero

```

Opuesto :: Dir -> Dir

Opuesto Norte = Sur

Opuesto Este = Oeste

Opuesto Sur = Norte

Opuesto _ = Este

IrAlBorde :: Dir -> GBoard -> GBoard

```
IrAlBorde d gb = if PuedoMover d gb
    then IrAlBorde d (Mover d gb)
    else gb
```

INV. REP.: Sea (GBB di hca mh hg) un GBoard

*Por cada HeadChange en la historia de hg debe haber un cambio registrado en la historia de hca

*Por cada registro de cambio en la historia de hca debe haber un HeadChange registrado en la historia hg

*Por cada ChangeCell en la historia de hg debe haber un cambio registrado en la historia del hca (Siendo hca = fromJust (lookupM c), siendo a su vez c una coordenada de un ChangeCell de hg) correspondiente a la coordenada del ChangeCell.

*Por cada valor de DC (siendo DC = domM mh) debe haber un ChangeCell en hg con el valor dado

*Por cada cambio registrado en la historia de un hca, debe haber un cambio registrado en la historia de hg de Change Cell Coord, donde Coord es la clave asociada a hcr en mh

*Ni el alto ni el ancho de ningún valor de dc puede superar ni el alto ni el ancho de Di

EmptyGBB :: Int -> Int -> GBoard

EmptyGBB al an = GBB (al,an) (newH (0,0)) emptyM (newH NoChange)

Poner :: Coord -> GBoard -> GBoard

```
Poner c (GBB co hc mh hg) = let Coord = Current hc
    in GBB co hc (assocM Coord (cambioHistoriaCelda c Coord mh))
    (Registrar (ChangeCru Coord) hg)
```

O(log N)

cambioHistoriaCelda :: Coord -> Coord -> Map k v -> History A

```
cambioHistoriaCelda Coord c co m = let nhc = lookupM co m
    in if (EsNothing mhc)
    then newH (PonerEn c (0000))
```

```

else Registrar (PonerEn c (Current (fromJust mhc)))
                    (fromJust mhc)

EsNothing :: Mayor A -> Bool
EsNothing Nothing = True
EsNothing _ = False

NroBolitas :: Coord -> GBBBoard -> Int
NroBolitas c (GBB _ hc m _) = let mhc = lookupM (current hc) m
                                in if (EsNothing mhc)
                                    then 0
                                    else NroBolitas c (current (fromJust mhc))

PuedeMover :: Dir -> GBBBoard -> Bool
PuedeMover d (GBB c hc _ _) = PuedeMoverA c d (Current hc)

Undo :: GBBBoard -> GBBBoard
Undo (GBB di hc mh hg) = let change = current hg
                          in if (EsHeadChange change)
                              then (GBB di (undo hc) mh (undo hg))
                              else (GBB di hc (UndoMpp change mh) (undo hg))

UndoMpp :: ChangeType -> Map k v -> Map k v
UndoMpp (ChangeCree c) m = let h = fromJust(lookupM c m)
                            in AssocM c (undo h) m

EsHeadChange :: ChangeType -> Bool
EsHeadChange HeadChange = True
EsHeadChange _ = False

```